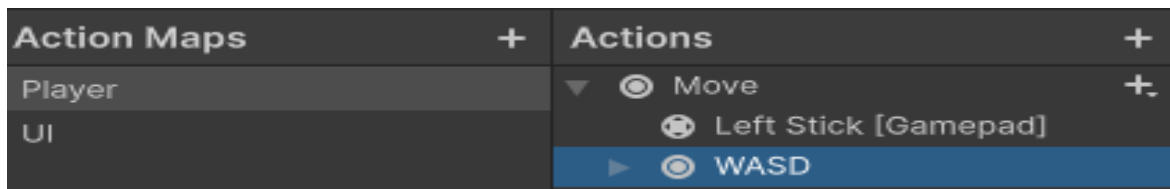




Devblog 1 - RollABall

Starting the tutorial section "Setting up the game" it was fairly simple to follow, I had to create the foundation by spawning a 3D object called a plane, which acted as the ground floor. Afterwards I also created a sphere which would later on in the tutorial become the player controlled object that we can control with "W,A,S,D". In this section we also created a materials folder where I colored my sphere to a bright shiny cyan.

Moving forward to the next part "Moving the Player" I learned that I could attach components to my sphere like adding a "Rigidbody" that would allow it to behave more like a real object for example calculating the physics, which will be required further ahead in the tutorial. We also added another component called "Player Input" which will assist in getting basic controls



working.

Then I started creating a custom script using C# called "PlayerController"

Within this script I learned that the method "Void Start" is called once on startup and the other method "Void Update" executes every frame, which depends on your "Frames Per Second" FPS. Now tying that to our "Player Input" component we can call "OnMove" to get information about our ball placement.

Even though method "OnMove" isn't called anywhere else within the script, it communicates with the "Player Input" component and updates our local variables "movementX, movementY" which then gets used within the "FixedUpdate" method to add force to the ball depending on "W,A,S,D", by utilizing the "Rigidbody" component attached to our sphere defined in the code as "rb"

```
void OnMove(InputValue movementValue)
{
    Vector2 movementVector = movementValue.Get<Vector2>();
    movementX = movementVector.x;
    movementY = movementVector.y;
}

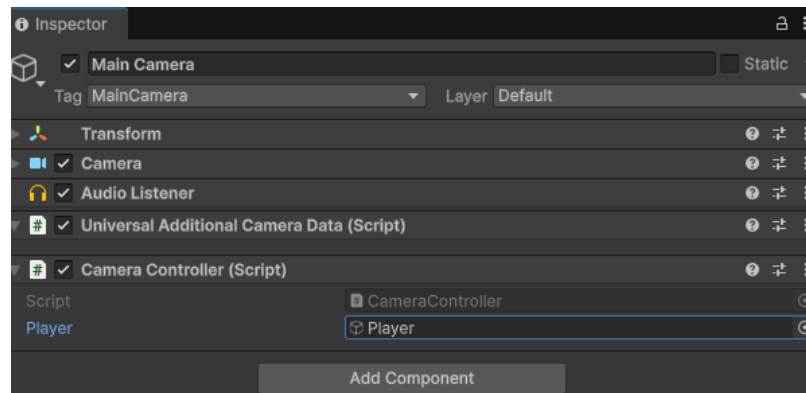
// Unity Message | 0 references
void FixedUpdate()
{
    // Move the player based on input
    Vector3 movement = new Vector3(movementX, 0f, movementY);
    rb.AddForce(movement * speed, ForceMode.Force);
}
```

```
void Start()
{
    rb = GetComponent<Rigidbody>();
    if (rb == null)
    {
        Debug.LogError("Missing Rigidbody on Player. Please add a Rigidbody component.");
        enabled = false;
        return;
    }
}
```

Name: Kim Kevin Harløv Madsen
University: VIA University College
Course: GMD 1



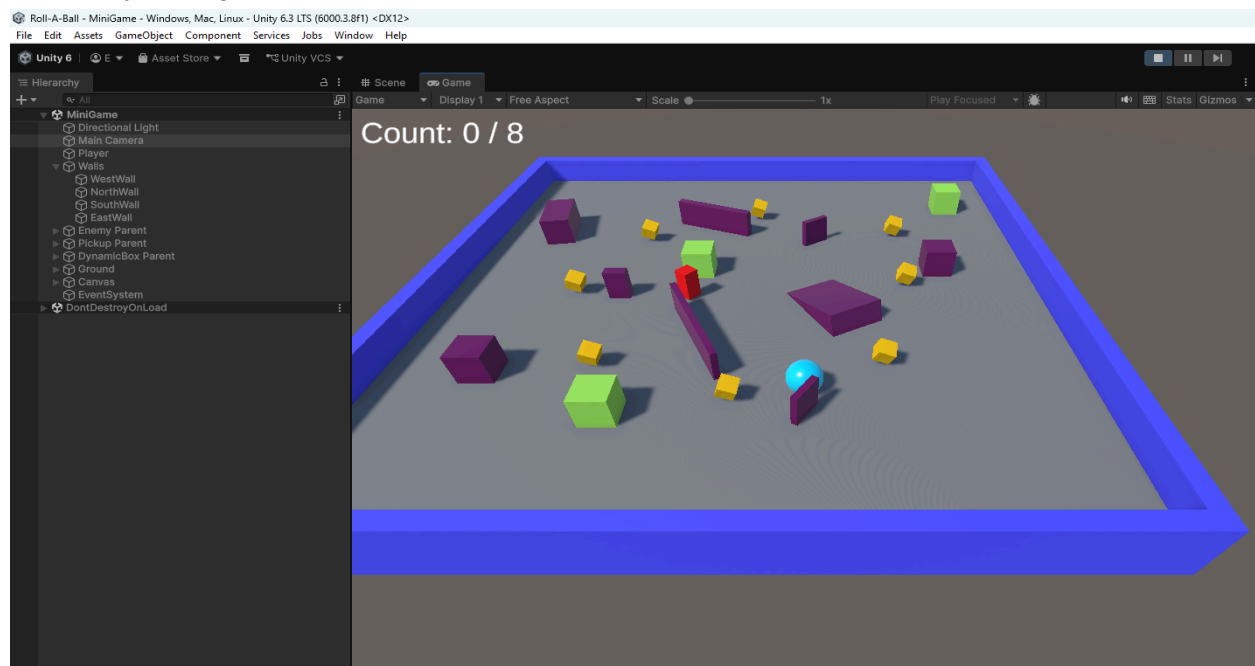
Moving on to the section “Moving the Camera” this required creating another custom script called “CameraScript” which had logic to follow the ball's movement at an angle. I just had to



drag the “Sphere/Player” gameobject into the script and it was all hooked up
In the next section “Setting up the Play Area” I just added walls around my “Plane” game object so that I couldn't roll my sphere off the edge into the void. Nothing special in this section.

Next I'll talk about sections “Creating Collectibles and Detecting Collisions with Collectibles” together. Within this section we had to create mini spheres that the player could collect to win the game. The requirements were that the cubes had an animation to rotate itself and also disappear once it collided with the player/sphere.

The final bits were about just wrapping up the game, having an enemy navigate the “NavMesh” and finally having a win/lose screen UI.



Github : [EQKim/Roll-A-Ball: GMD1 Assignment 1](https://github.com/EQKim/Roll-A-Ball-GMD1-Assignment-1)

Github Main Project : [GitHub - EQKim/GMD1: GMD1 - VIA · GitHub](https://github.com/EQKim/GMD1-VIA)